



EIGER Data Management Standards, Database Access Instructions and Fileserver Documentation

[illegible]

1. Purpose

2. Configuring Database Access using AWS Credentials

High-level data products about EIGER fields are stored in a SQL relational database hosted on the Amazon cloud via the Amazon Web Services (AWS) Relational Database Service (RDS). The database is configured in standard Postgresql format, to facilitate communication with a custom python-based API developed by the project team. Raw data products in fits format are also stored on the cloud using Amazon's S3 service. By storing certified project data products on a single, robust cloud server, we eliminate the need to synchronize versions of data products, match reduction runs, and increase visibility and transparency across the geographically distributed team.

Each user's AWS credentials are configured in a way that avoids the need to manage and enter passwords constantly, but it does take a small amount of initial work to set up. We use the Amazon "IAM" credential service, along with Amazon's boto3 python API to build in automatic authentication from python clients. Users install keys on their local host, and then boto3 calls a function that interacts with AWS to generate secure, temporary passwords with very short expiration periods (~10-20 minutes) and automatically uses them to log in without interaction by the user. This section documents steps to set this up on the client side.

2.1. Obtain your credentials

Contact the DB administrator to obtain two keys needed for login: the

"AWS Access Key ID" and the

"AWS Secret Access Key"

These are NOT the same thing. They should already have been generated when your username/account were created.

Note that the secret key is only made visible one time by AWS when the account is created by the administrator. If you lose your secret key, contact the DB administrator. Your lost key will need to be removed, and a new one can be generated.

2.2. Install Amazon Command Line Interface (CLI) tools

Local copies of the keys must be installed on your machine using Amazon's CLI interface, which you must download and install. Full instructions can be found at [this link](#). Many of our group members will be installing locally on a Mac laptop, and the following commands should download and then install (the ">" is a terminal prompt):

```
> curl "https://awscli.amazonaws.com/AWSCLIV2.pkg" -o "AWSCLIV2.pkg"
> sudo installer -pkg AWSCLIV2.pkg -target /
```

The second command requires root privileges on your laptop or local machine.

2.3. *Configure your local environment*

After you have installed the AWS CLI tools (Section 2.2), you need to register the keys obtained from the administrator. From a Terminal prompt, enter:

```
> aws configure
```

You will get several prompts, which should be filled in as follows:

```
AWS Access Key ID [None]: <enter AWS Access Key ID here>
AWS Secret Access Key [None]: <enter AWS Secret Access Key here>
Default region name [None]: us-east-2
Default output format [None]: json
```

The last two fields should be entered as shown when prompted. We may be able to create a European access region if there are connection lags. The default output should be json for compatibility with the eager python code.

This will create a directory ~/.aws containing some configuration information, and send additional information to AWS to configure the remote end of the connection. You should now be ready to connect to the database.

2.4. *Install relevant python libraries*

You will need to install the AWS API python library, called boto3:

```
> python -m pip install boto3
```

And also a python library to handle postgresql database connections. We will use the psycopg2 library, which is the most widely-used community standard.

```
> pip install psycopg2
```

Some team members have had more success installing psycopg2 using conda rather than pip, both are supported.

2.5. *Test the connection using provided sample script*

From a terminal prompt, cd into your [EIGER Git directory]/Database/SQL. Open the file eager_dbtest.py in an editor, and change the field “USR” to your username for the database. Everything else should remain the same.

Then run:

```
> cd [EIGER GitHub directory]/Database/SQL
> python "eager_dbtest.py"
```

If the code worked correctly, it should print out a list of the quasars to be targeted with JWST.

2.6. Test credentials with the AWS Fileserver S3

Large fits data files are not stored in the SQL database, as this is more efficient for storing tabulated numerical data than large binary files. It is more convenient to store these on a cloud fileserver, in this case the AWS S3 system. The IAM credentials you set up earlier should also give you access to a special gto1243 file server area on S3.

To test your access, you can run the following:

```
> cd [EIGER GitHub directory]/Database/Fileserver
> python s3_test.py
```

And you should see a printout as follows:

```
[{'Name': 'gto1243', 'CreationDate': datetime.datetime(2021, 6, 2, 2, 50, 31, tzinfo=tzutc())}]
```

This is the GTO1243 file server, named after the EIGER's JWST program handle.

Also, upon success the code will download a FIRE spectrum of ULAS1120 (in fits format) to your working directory from the S3 Fileserver. This file can be deleted after the test is run.